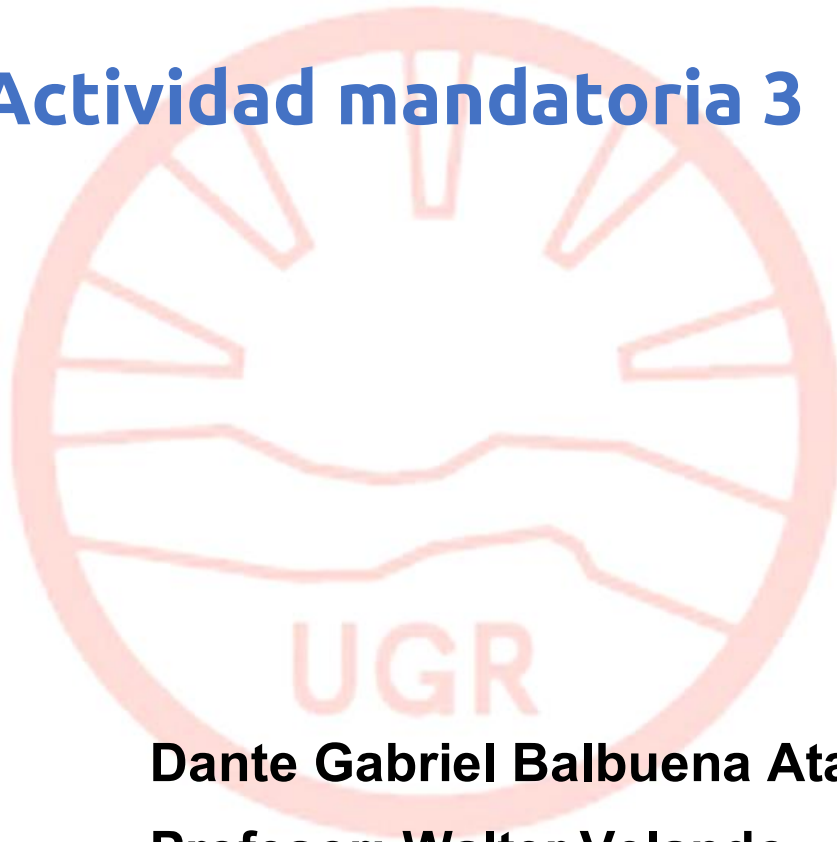


TECNICATURA UNIVERSITARIA EN CIBERSEGURIDAD

ISEGURIDAD EN CLOUD SERVICES

Actividad mandatoria 3



Dante Gabriel Balbuena Atar

Profesor: Walter Velando

Tucumán, Tafí Viejo – 13/11/2025

Contenido

1. INTRODUCCIÓN	3
2. DESARROLLO	3
2.1 EVALUACIÓN DE LA INFRAESTRUCTURA ACTUAL.....	3
VULNERABILIDAD V1: S3 MAL CONFIGURADO (EXPOSICIÓN DE DATOS	
SENSIBLES)	3
✓ ¿Qué está pasando?	3
✓ ¿Por qué es grave?	3
✓ Causa raíz:	4
PROPUESTAS DE MEJORA	4
1. Aplicar S3 Block Public Access a nivel de cuenta	4
2. Crear bucket policies restrictivas	4
3. AWS Config Rules para auditoría continua	5
4. AWS Macie	5
VULNERABILIDAD V2: FALTA DE INSPECCIÓN DEL TRÁFICO ENTRE	
SUBREDES (RIESGO DE MOVIMIENTO LATERAL)	5
✓ ¿Qué está pasando?	5
✓ ¿Por qué es grave?	5
✓ Causa raíz:	6
PROPUESTAS DE MEJORA	6
1. Implementar AWS Network Firewall	6
2. Usar VPC Traffic Mirroring	6
3. Habilitar VPC Flow Logs (REQUIRED).....	6
4. Activar Amazon GuardDuty.....	7
TABLA: VULNERABILIDAD → CAUSA → ACCIÓN CORRECTIVA.....	7
2.1.1 ARQUITECTURA PROPUESTA INTEGRADA	7
2.2 GESTIÓN DE IDENTIDAD Y ACCESO (IAM)	8

VULNERABILIDAD 1: PERMISOS EXCESIVOS EN USUARIOS, ROLES Y

POLÍTICAS IAM.....	8
--------------------	---

Descripción del problema	9
--------------------------------	---

Riesgos.....	9
--------------	---

Acciones correctivas propuestas.....	9
--------------------------------------	---

VULNERABILIDAD 2: CUENTAS ADMINISTRATIVAS SIN MFA HABILITADO. 9	
---	--

Descripción del problema	10
--------------------------------	----

Riesgos.....	10
--------------	----

Acciones correctivas propuestas.....	10
--------------------------------------	----

TABLA PROFESIONAL DE VULNERABILIDADES IAM	11
---	----

2.3 DESARROLLO SEGURO DE APLICACIONES (DEVSECOPS).....	11
--	----

2.3.1 Evaluación del Proceso de Desarrollo Actual	11
---	----

1. Uso de credenciales Hardcoded en repositorios	11
--	----

2. Falta de validación de entradas en el backend	11
--	----

3. Ausencia de un pipeline de CI/CD seguro	12
--	----

4. Falta de límites y controles en el proceso de despliegue	12
---	----

2.3.2 PROPUESTA DE MARCO DEVSECOPS	12
--	----

✓ 1. Seguridad en el Código.....	12
----------------------------------	----

a) Eliminación de credenciales hardcoded	12
--	----

b) Validación de entradas.....	13
--------------------------------	----

✓ 2. Seguridad en el Pipeline CI/CD.....	13
--	----

SAST (Static Application Security Testing).....	13
---	----

SCA (Software Composition Analysis)	13
---	----

DAST (Dynamic Analysis Security Testing)	13
--	----

Escaneo de contenedores	14
-------------------------------	----

Seguridad en despliegue	14
-------------------------------	----

✓ 3. Seguridad en la Ejecución	14
--------------------------------------	----

WAF + Reglas OWASP	14
--------------------------	----

API Gateway como barrera de validación	14
--	----

✓ 4. Monitoreo y Alertas de Seguridad	15
---	----

Activar:	15
2.3.3 SOLUCIONES A LAS PRÁCTICAS INSEGURAS DETECTADAS	15
2.4 PRUEBAS DE SEGURIDAD Y ANÁLISIS DE VULNERABILIDADES.....	15
2.4.1 EVALUACIÓN DEL ESTADO ACTUAL DE PRUEBAS DE SEGURIDAD ...	16
1. Ausencia de pruebas automáticas de seguridad en CI/CD	16
2. Falta de pruebas manuales especializadas	16
3. No existe un inventario de vulnerabilidades	17
4. No hay pruebas de seguridad antes de cada release	17
2.4.2 PROPUESTA DE PROCESO INTEGRAL DE PRUEBAS DE SEGURIDAD .	17
A. PRUEBAS AUTOMÁTICAS (INTEGRACIÓN AL CI/CD)	17
✓ 1. SAST – Análisis Estático del Código	17
✓ 2. SCA – Escaneo de Dependencias.....	18
✓ 3. DAST – Análisis Dinámico	18
✓ 4. Escaneo de Contenedores (si aplica)	19
B. PRUEBAS MANUALES PERIÓDICAS	19
✓ 1. Pentesting trimestral.....	19
✓ 2. Red Team anual.....	19
✓ 3. Revisiones de código seguras (manuales).....	20
C. GESTIÓN DE VULNERABILIDADES	20
✓ 1. Identificación.....	20
✓ 2. Clasificación.....	20
✓ 3. Asignación	21
✓ 4. Remediación.....	21
✓ 5. Verificación.....	21
✓ 6. Reporte.....	21
2.4.3 HERRAMIENTAS RECOMENDADAS ESPECÍFICAS DE AWS	21
3. DOCUMENTACIÓN Y PLAN DE IMPLEMENTACIÓN.....	22

3.1 RESUMEN DE HALLAZGOS CRÍTICOS	22
Infraestructura	22
Gestión de Identidad (IAM)	22
Aplicaciones en la nube	22
3.2 OBJETIVOS DEL PLAN DE IMPLEMENTACIÓN	23
3.3 PLAN DE IMPLEMENTACIÓN DETALLADO	23
FASE 1: ENDURECIMIENTO DE INFRAESTRUCTURA (SEMANA 1 A 2)	23
1. Seguridad en almacenamiento (S3)	23
2. Seguridad de red	23
FASE 2: GESTIÓN DE IDENTIDADES Y GOBERNANZA (SEMANA 2 A 4)	24
1. Reestructuración de IAM	24
2. Fortalecimiento de acceso	24
3. Gobernanza con AWS Organizations	24
FASE 3: SEGURIDAD DE APLICACIONES (SEMANA 4 A 8)	24
1. Migración a DevSecOps	24
2. Eliminación de credenciales hardcodeadas	25
3. Validación segura de entradas	25
FASE 4: DETECCIÓN Y RESPUESTA CONTINUA (SEMANA 8 EN ADELANTE)	
25	
Monitoreo y detección	25
Alertas	25
Respuesta a incidentes	25
3.4 ESTRATEGIA DE PRIORIDAD (PARA JUSTIFICAR ANTE PROFESOR)	26
3.5 RESULTADO ESPERADO	26
3. CONCLUSIONES	26

1. Introducción

En este trabajo se analiza la infraestructura de una empresa de servicios en la nube que enfrenta un incremento en incidentes de seguridad. El objetivo es diseñar una arquitectura más segura y resiliente mediante la adopción de controles de seguridad en múltiples capas, desde la infraestructura hasta el desarrollo de aplicaciones. **Se elige AWS** principalmente porque tengo más experiencia usando este servicio y por su madurez en herramientas de seguridad, escalabilidad y gobierno de identidades, así como su integración con servicios de monitoreo y cumplimiento normativo.

2. Desarrollo

2.1 Evaluación de la Infraestructura Actual

La empresa opera sobre una arquitectura basada en **AWS**, con una **VPC (Virtual Private Cloud)** organizada en subredes públicas y privadas.

Esta segmentación es correcta desde un punto de vista de diseño, pero las configuraciones actuales presentan vulnerabilidades que comprometen **confidencialidad, integridad y disponibilidad** de los datos y servicios.

Vulnerabilidad V1: S3 MAL CONFIGURADO (EXPOSICIÓN DE DATOS SENSIBLES)

✓ ¿Qué está pasando?

Algunos buckets de Amazon S3 permiten acceso público o semipúblico por configuraciones incorrectas:

- *Block Public Access* desactivado.
- Políticas con "Principal": "*"
- Archivos accesibles mediante URL pública.
- Falta de cifrado en reposo y en tránsito.

✓ ¿Por qué es grave?

S3 es utilizado para almacenar:

- logs de aplicación
- respaldos
- archivos subidos por usuarios (incluyendo PII o datos financieros)

Una mala configuración permite:

- fuga de datos
- cumplimiento normativo violado (ISO 27018, GDPR, PCI)
- exposición a ransomware
- manipulación de información

✓ Causa raíz:

1. Falta de gobernanza centralizada sobre S3.
2. Falta de revisiones automáticas de seguridad.
3. Ausencia de control de acceso granular (IAM y policies).

Propuestas de Mejora

1. Aplicar *S3 Block Public Access* a nivel de cuenta

Esto garantiza que NINGÚN bucket pueda exponerse accidentalmente:

- `BLOCKPUBLICACLS = TRUE`
- `BLOCKPUBLICPOLICY = TRUE`
- `RESTRICTPUBLICBUCKETS = TRUE`

Justificación técnica:

Evita que una política mal aplicada permita acceso público aunque esté dentro de una VPC privada.

2. Crear *bucket policies* restrictivas

Usar políticas basadas en condiciones:

Ejemplo:

```
{
  "EFFECT": "DENY",
  "PRINCIPAL": "*",
  "ACTION": "S3:*",
  "RESOURCE": ["ARN:AWS:S3:::EMPRESA-SEGURA/*"],
  "CONDITION": {
    "BOOL": {"AWS:SECURETRANSPORT": "FALSE"}
  }
}
```

- ✓ Obliga a usar HTTPS
- ✓ Evita accesos de redes no autorizadas
- ✓ Evita que un usuario interno suba datos por HTTP

3. AWS Config Rules para auditoría continua

Reglas recomendadas:

Regla	Qué detecta
s3-bucket-public-read-prohibited	buckets expuestos a lectura pública
s3-bucket-public-write-prohibited	escritura pública
s3-bucket-server-side-encryption-enabled	cifrado obligatorio
s3-bucket-logging-enabled	auditoría habilitada

→ Si alguna regla falla → la cuenta queda en **estado no conforme**.

4. AWS Macie

Macie escanea los buckets y detecta automáticamente:

- **DNI**
- **TARJETAS DE CRÉDITO**
- **CONTRASEÑAS**
- **INFORMACIÓN PERSONAL (PII)**
- **DOCUMENTOS SENSIBLES**

Justificación técnica:

Permite identificar datos críticos en un bucket antes de limpiarlo o moverlo.

Vulnerabilidad V2: Falta de inspección del tráfico entre subredes (riesgo de movimiento lateral)

✓ ¿Qué está pasando?

Hay tráfico entre subredes públicas ↔ privadas que no está inspeccionado.

Ejemplos:

- EC2 públicas pueden comunicarse innecesariamente con bases de datos.
- No hay firewall de capa 4/7 entre subredes.
- No se registran paquetes sospechosos.
- No hay IDS/IPS interno.

✓ ¿Por qué es grave?

Si un servidor en la subred pública es comprometido, un atacante puede:

- moverse lateralmente
- escanear puertos internos
- acceder a RDS o servicios internos
- capturar credenciales

Esto es exactamente el tipo de ataque que MITRE describe como **T1021 (Lateral Movement)**.

✓ Causa raíz:

- Security Groups permisivos
- Network ACLs basadas en allow/any
- No hay firewall administrado
- No hay segmentación por función (solo por “pública” y “privada”)

Propuestas de Mejora

1. Implementar AWS Network Firewall

Permite inspección profunda (DPI), filtrado y reglas IDS/IPS internas:

- bloquear tráfico no autorizado
- detectar conexiones sospechosas
- inspeccionar patrones de ataque (OWASP, malware, escaneos)

Arquitectura recomendada:

Public Subnet → Firewall Subnet → Private Subnet

2. Usar VPC Traffic Mirroring

Permite duplicar tráfico hacia una herramienta de análisis (Suricata, Zeek, etc.).

Ejemplo práctico:

- espejo del tráfico de EC2 públicas
 - enviado a un “analysis instance”
 - detecta patrones anómalos

3. Habilitar VPC Flow Logs (REQUIRED)

Captura metadatos de cada conexión aceptada/denegada.
Integración con:

- CloudWatch Logs
- S3
- GuardDuty

Permite detectar:

- puertos escaneados
- conexiones internas inusuales
- acceso inesperado entre subredes

4. Activar Amazon GuardDuty

Detecta:

- comportamiento malicioso
- exfiltración de datos
- actividad desde IPs comprometidas
- movimiento lateral
- instancias EC2 con malware o bots

Tabla: Vulnerabilidad → Causa → Acción Correctiva

Vulnerabilidad	Causa específica	Acción recomendada
S3 expuesto	políticas permisivas, ausencia de auditoría	S3 Block Public Access, Config Rules, Macie
Falta de inspección interna	sin firewall interno, SG abiertos	Network Firewall, Traffic Mirroring, Flow Logs
Permisos excesivos IAM	políticas *.*	Principio de menor privilegio
Cuentas sin MFA	controles de acceso débiles	MFA obligatorio, IAM Password Policy

2.1.1 Arquitectura Propuesta Integrada

Para complementar la evaluación de la infraestructura actual y las mejoras recomendadas, se presenta la siguiente arquitectura multi-región diseñada bajo

principios de seguridad, alta disponibilidad y resiliencia.

La propuesta integra dos regiones de AWS (us-east-1 como región primaria y us-west-2 como secundaria), incorpora balanceo global mediante Amazon Route 53, distribución a través de Amazon CloudFront, replicación cruzada de objetos S3, bases de datos distribuidas (Aurora Global/DynamoDB Global) y controles de seguridad nativos como IAM, KMS, AWS WAF y Shield Advanced.

A continuación se detalla el diagrama general de la solución:

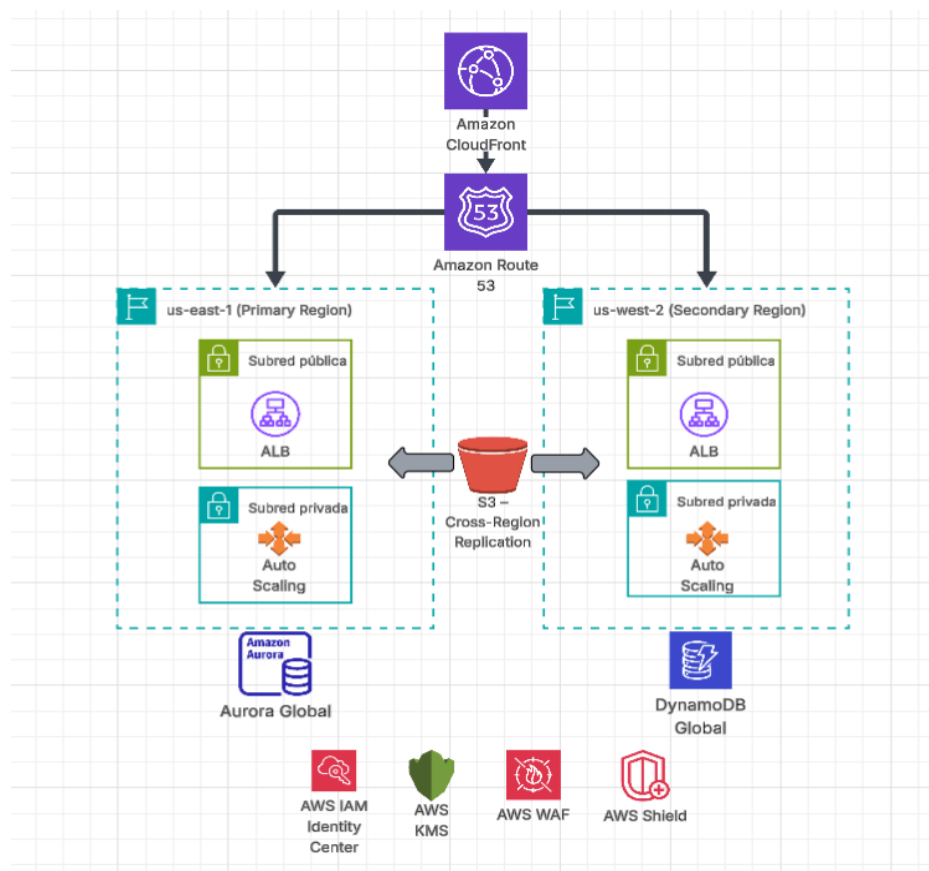


Imagen 1 Diagrama general

2.2 Gestión de Identidad y Acceso (IAM)

(Versión profunda, clara y con ejemplos técnicos — lista para poner en el TP)

La gestión de identidad y acceso es uno de los componentes más críticos de seguridad en la nube. En la infraestructura actual se identificaron vulnerabilidades asociadas a permisos excesivos y mecanismos deficientes de autenticación, lo que aumenta la superficie de ataque y habilita la posibilidad de escalación de privilegios en caso de compromiso.

Vulnerabilidad 1: Permisos excesivos en usuarios, roles y políticas IAM

Descripción del problema

Se detectó que varios usuarios y roles tienen políticas del tipo:

```
{  
  "Effect": "Allow",  
  "Action": "*",  
  "Resource": "*"  
}
```

Esto otorga **permisos administrativos totales y sin restricciones**.

Este tipo de configuraciones es común en etapas de desarrollo, pero en entornos productivos representan una amenaza severa.

Riesgos

- Escalamiento total si un atacante compromete una cuenta.
- Modificación o eliminación de recursos críticos (VPC, S3, RDS).
- Acceso a datos sensibles o secretos.
- Creación de nuevas llaves o roles maliciosos.

Acciones correctivas propuestas

1. **Aplicación estricta del principio de menor privilegio**
Crear políticas personalizadas basadas en tareas ("job-based policies").
Ejemplo: un desarrollador solo necesita acceso a CloudWatch Logs y Lambda Deploy, no a IAM ni EC2.
2. **Reemplazar políticas administradas por el usuario**
Sustituir las "inline policies" peligrosas por AWS Managed Policies o políticas administradas por la organización.
3. **Implementar revisiones periódicas de permisos (IAM Access Analyzer)**
Configurar para detectar:
 - a. recursos accesibles desde cuentas externas (cross-account)
 - b. políticas con permisos excesivos
 - c. roles con trust policy demasiado amplia
4. **Rotación automática de claves y eliminación de claves no usadas**
AWS IAM → "Access Key Last Used" para identificar credenciales obsoletas.

Vulnerabilidad 2: Cuentas administrativas sin MFA habilitado

Descripción del problema

Se detectó que usuarios con privilegios administrativos (IAM Users con AdministratorAccess o acceso a consola) **no tienen MFA configurado**, lo cual expone la organización a compromisos por:

- contraseñas débiles
- phishing
- brechas externas
- reutilización de contraseñas

Riesgos

- Acceso total con un solo vector (password leak).
- Imposibilidad de rastrear accesos maliciosos.
- Compromiso completo de la infraestructura.

Acciones correctivas propuestas

1. **Habilitar MFA obligatorio en todos los usuarios con acceso a consola**
Idealmente usar:
 - a. FIDO2 keys (YubiKey)
 - b. Authenticator apps (Microsoft Authenticator / Google Authenticator)
2. **Eliminar IAM Users cuando sea posible → reemplazar por IAM Roles + SSO**
IAM Identity Center (AWS SSO) permite:
 - a. autenticación centralizada
 - b. MFA forzado
 - c. sesiones temporales
 - d. auditoría más clara
3. **Aplicar AWS Organizations + Service Control Policies (SCP)**
Crear una política tipo:

```
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "BoolIfExists": {"aws:MultiFactorAuthPresent": "false"}
  }
}
```

Esto **prohíbe cualquier acción de alto impacto** si el usuario no tiene MFA.

Tabla profesional de vulnerabilidades IAM

Vulnerabilidad	Causa raíz	Riesgo	Acción correctiva
Permisos excesivos (IAM)	Políticas *.*, falta de rol por tarea	Escalamiento total, acceso a datos	Menor privilegio, IAM Access Analyzer, políticas personalizadas
Cuentas sin MFA	Mala higiene de credenciales	Compromiso total con un password	

2.3 Desarrollo Seguro de Aplicaciones (DevSecOps)

La empresa actualmente desarrolla aplicaciones en la nube sin integrar controles de seguridad dentro del ciclo de vida del desarrollo. Esto aumenta la probabilidad de introducir vulnerabilidades desde etapas tempranas del proceso, lo que se traduce en riesgos como exposición de credenciales, fallas de validación de entrada, dependencias vulnerables o configuraciones inseguras.

A continuación se presenta una evaluación del estado actual y una propuesta integral de adopción de un modelo **DevSecOps** que garantice seguridad continua.

2.3.1 Evaluación del Proceso de Desarrollo Actual

Se identificaron las siguientes prácticas inseguras:

1. Uso de credenciales Hardcoded en repositorios

Los desarrolladores almacenan claves de acceso, tokens o passwords directamente en el código fuente.

Esto expone a la empresa a:

- robo de credenciales,
- accesos no autorizados a bases de datos,
- manipulación de entornos productivos.

Incluso repositorios privados pueden ser comprometidos si un atacante obtiene acceso

por phishing o credenciales filtradas.

2. Falta de validación de entradas en el backend

Las aplicaciones no incluyen sanitización y validación estricta de parámetros enviados por los usuarios.

Esto abre la puerta a ataques como:

- **SQL Injection,**
- **Command Injection,**
- **Cross-Site Scripting (XSS),**
- **Deserialización insegura.**

3. Ausencia de un pipeline de CI/CD seguro

No existen:

- análisis estáticos (SAST),
- análisis dinámicos (DAST),
- escaneo de dependencias (SCA),
- escaneo de contenedores si se usan Docker/ECS/EKS.

Esto permite que código vulnerable llegue a producción sin detección previa.

4. Falta de límites y controles en el proceso de despliegue

Cualquier desarrollador puede desplegar a producción, lo que aumenta el riesgo de configuraciones incorrectas, versiones inseguras o despliegues accidentales.

2.3.2 Propuesta de Marco DevSecOps

Se propone implementar un enfoque donde **la seguridad se integre desde la etapa de codificación hasta la operación.**

El pipeline seguro incluye cuatro fases principales:

✓ 1. Seguridad en el Código

a) Eliminación de credenciales hardcoded

Todo secreto debe administrarse con:

- **AWS Secrets Manager**
- AWS Systems Manager **Parameter Store** (para configuraciones no sensibles)

Implementaciones:

- Variables de entorno con rotación automática.
- IAM Roles para evitar uso de claves estáticas.
- Políticas que impiden push si se detectan secretos (pre-commit).

b) Validación de entradas

Usar:

- Validación server-side con librerías seguras (p. ej., Joi, Marshmallow, Express-validator).
- Sanitización automática.
- Schemas OpenAPI con API Gateway para validar requests.

✓ 2. Seguridad en el Pipeline CI/CD

Agregar herramientas automáticas en cada fase:

SAST (Static Application Security Testing)

Analiza el código antes del build.

Ejemplos:

- SonarQube
- Snyk Code
- GitHub Advanced Security

Detecta:

- SQL Injection,
- hardcoded passwords,
- flujos inseguros.

SCA (Software Composition Analysis)

Analiza vulnerabilidades en dependencias (NPM, pip, Maven).

Herramientas:

- OWASP Dependency-Check
- Snyk
- BlackDuck

DAST (Dynamic Analysis Security Testing)

Pruebas de seguridad contra la aplicación ejecutándose.

Herramientas:

- OWASP ZAP
- Burp Suite automated

Detecta:

- XSS,
- CSRF,
- errores de configuración.

Escaneo de contenedores

Si se usan contenedores:

- Amazon Inspector para ECR
- Trivy
- Snyk Container

Detectan vulnerabilidades en imágenes.

Seguridad en despliegue

- Firmado de artefactos.
- Revisión obligatoria (code review).
- Deploy automático solo si todas las verificaciones pasan.

✓ 3. Seguridad en la Ejecución

WAF + Reglas OWASP

AWS WAF protege contra:

- inyección,
- XSS,
- bots maliciosos.

API Gateway como barrera de validación

Se recomienda:

- limitar requests,
- validar estructuras JSON (schema validation),
- aplicar throttling para evitar abusos.

✓ 4. Monitoreo y Alertas de Seguridad

Activar:

- AWS CloudWatch (errores en aplicaciones)
- AWS GuardDuty (actividad sospechosa)
- AWS CloudTrail (auditoría de llamadas API)
- AWS X-Ray (trazabilidad del backend)

Estas herramientas permiten detectar comportamientos anómalos, tráfico irregular o uso indebido de credenciales.

2.3.3 Soluciones a las prácticas inseguras detectadas

Vulnerabilidad	Causa	Riesgo	Solución propuesta
Credenciales hardcoded	Falta de gestión de secretos	Robo de datos / acceso no autorizado	Secrets Manager + IAM Roles
Falta de validación	Código sin sanitización	Inyección, XSS	Validación servidor + WAF
Dependencias vulnerables	No escaneo	Ejecución de librerías inseguras	SCA automático
Pipeline sin seguridad	CI/CD sin controles	Subida de código vulnerable	SAST + DAST + firmar artefactos
Cualquier dev despliega	Falta de gobernanza	Cambios inseguros en prod	Revisión obligatoria + permisos restringidos

2.4 Pruebas de Seguridad y Análisis de Vulnerabilidades

La empresa actualmente no cuenta con un proceso formal y continuo de pruebas de

seguridad sobre sus aplicaciones en la nube. Esto genera una ventana de exposición que permite que vulnerabilidades lleguen a producción sin ser detectadas, facilitando ataques como inyecciones, escalamiento de privilegios, robo de credenciales o ejecución remota de código.

A continuación se presenta un análisis del estado actual y un plan de remediación con herramientas concretas.

2.4.1 Evaluación del Estado Actual de Pruebas de Seguridad

Tras la auditoría inicial se identificaron las siguientes deficiencias:

1. Ausencia de pruebas automáticas de seguridad en CI/CD

Actualmente no se realizan:

- **SAST** (Static Application Security Testing)
- **DAST** (Dynamic Application Security Testing)
- **SCA** (Software Composition Analysis)
- **Escaneo de contenedores** (si la empresa usa Docker/ECS)

Esto significa que:

- vulnerabilidades ingresan desde el código hacia producción,
- dependencias desactualizadas con CVEs pueden ejecutarse en servidores,
- no existe validación dinámica contra ataques comunes.

2. Falta de pruebas manuales especializadas

El equipo no realiza:

- Ethical hacking interno,
- pruebas de penetración (pentesting) periódico,
- análisis de lógica de negocio.

Esto limita la detección de fallas complejas, como:

- bypass de autenticación,
- manipulación de precios,
- lógica vulnerable en flujos críticos.

3. No existe un inventario de vulnerabilidades

La compañía no mantiene un registro centralizado de:

- vulnerabilidades detectadas,
- severidad (CVSS),
- fecha de hallazgo,
- estado (abierto/cerrado),
- responsables,
- plazos de remediación.

Esto dificulta medir riesgo real.

4. No hay pruebas de seguridad antes de cada release

Los despliegues se basan en pruebas funcionales, sin controles de seguridad obligatorios.

Esto incrementa la probabilidad de introducir configuraciones inseguras en producción.

2.4.2 Propuesta de Proceso Integral de Pruebas de Seguridad

A continuación se detalla un modelo completo que debe integrarse al pipeline DevSecOps.

A. Pruebas Automáticas (Integración al CI/CD)

Se implementan herramientas automáticas que se ejecutan en cada push o PR (pull request):

✓ 1. SAST – Análisis Estático del Código

Detecta vulnerabilidades en el código antes del build.

Herramientas recomendadas:

- **SonarQube**
- **GitHub Advanced Security**
- **Snyk Code**

Detecta:

- SQL injection
- hardcoded secrets
- accesos inseguros
- fallas de validación
- malas prácticas criptográficas

✓ 2. SCA – Escaneo de Dependencias

Detecta vulnerabilidades presentes en librerías externas.

Herramientas:

- OWASP Dependency-Check
- Snyk
- BlackDuck

Esto protege contra:

- CVEs conocidos
- librerías abandonadas
- paquetes manipulados o comprometidos

✓ 3. DAST – Análisis Dinámico

Se prueba la aplicación en ejecución, como lo haría un atacante.

Herramientas:

- OWASP ZAP
- Burp Suite (modo automated)

Detecta:

- XSS
- CSRF
- inyección
- fugas de información

- errores de configuración

✓ 4. Escaneo de Contenedores (si aplica)

Para repositorios ECR se recomienda:

- **Amazon Inspector**
- Trivy
- Snyk Container

Buscan:

- imágenes vulnerables
- paquetes obsoletos
- configuraciones inseguras

B. Pruebas Manuales Periódicas

Se complementan con pruebas realizadas por analistas humanos.

✓ 1. Pentesting trimestral

Se realiza un pentest interno o contratado a terceros para evaluar:

- APIs
- backend
- front
- autenticación
- infraestructura asociada

Incluye:

- explotación real
- búsqueda de fallas lógicas
- revisión de permisos IAM
- evasión de WAF

✓ 2. Red Team anual

Simulación de ataque avanzada (opcional si querés sumar puntos extra).

Evalúa:

- resiliencia
- detección temprana
- respuesta a incidentes

✓ 3. Revisiones de código seguras (manuales)

Especialistas revisan:

- mal uso de criptografía
- autenticación insegura
- flujos de permisos
- entrada de usuario compleja

C. Gestión de Vulnerabilidades

Se implementa un ciclo formal basado en:

✓ 1. Identificación

Los hallazgos entran por:

- CI/CD
- pentesting
- monitoreo
- informes de desarrolladores

✓ 2. Clasificación

Se asigna severidad con **CVSS v3.1**:

- Crítica
- Alta
- Media
- Baja

✓ 3. Asignación

Cada vulnerabilidad tiene:

- responsable
- fecha límite
- evidencia técnica

✓ 4. Remediación

Las correcciones deben pasar por:

- revisión de código
- pruebas automáticas
- validación DAST posterior

✓ 5. Verificación

Se confirma que la vulnerabilidad fue solucionada sin introducir nuevas fallas.

✓ 6. Reporte

Se mantiene un tablero (Jira, Notion, Security Hub) con:

- total de vulnerabilidades activas
- severidad
- tiempo promedio de remediación (MTTR)
- cumplimiento

2.4.3 Herramientas Recomendadas Específicas de AWS

Para alinearte con la consigna (usar un proveedor elegido), se recomienda incluir:

- **AWS CodePipeline + CodeBuild** → automatizar SAST, SCA, DAST
- **Amazon Inspector** → escaneo continuo
- **AWS WAF** → protección web
- **AWS Security Hub** → tablero centralizado
- **Amazon GuardDuty** → detección de actividad anómala
- **CloudTrail** → auditoría de APIs

- **CloudWatch** → logs y alertas

Esto muestra conocimiento aplicado y concreto del entorno AWS.

3. Documentación y Plan de Implementación

Esta sección resume todos los hallazgos y define un plan concreto, ordenado y ejecutable para implementar las mejoras de seguridad propuestas en la infraestructura, en IAM y en las aplicaciones de la empresa.

3.1 Resumen de Hallazgos Críticos

Tras la auditoría completa se identificaron las siguientes áreas críticas de riesgo:

Infraestructura

Vulnerabilidad	Descripción	Riesgo
S3 mal configurado	Buckets con acceso público o sin cifrado obligatorio	Filtración de datos sensibles
Falta de inspección entre subredes	No existe firewall interno ni monitoreo profundo	Movimiento lateral y persistencia

Gestión de Identidad (IAM)

Vulnerabilidad	Descripción	Riesgo
Permisos excesivos	Roles con políticas *.* y privilegios amplios	Escalamiento de privilegios
Cuentas sin MFA	Accesos privilegiados sin segundo factor	Compromiso total ante robo de credenciales

Aplicaciones en la nube

Vulnerabilidad	Descripción	Riesgo
Credenciales hardcodeadas	Claves expuestas en código / repos	Robo de claves y acceso no autorizado
Falta de validación	No se sanitizan parámetros	Inyección, XSS, manipulación de datos

No existe DevSecOps	Sin pruebas SAST, DAST, SCA	Vulnerabilidades ingresan al pipeline
---------------------	-----------------------------	---------------------------------------

3.2 Objetivos del Plan de Implementación

1. Reducir la superficie de ataque en almacenamiento, redes e identidades.
2. Incorporar seguridad automatizada en el ciclo de desarrollo.
3. Crear una arquitectura resiliente y gobernada.
4. Implementar controles continuos para detección y respuesta a incidentes.

3.3 Plan de Implementación Detallado

El plan se divide en fases claras. Esto muestra profesionalismo y planificación real.

Fase 1: Endurecimiento de Infraestructura (Semana 1 a 2)

1. Seguridad en almacenamiento (S3)

- Aplicar políticas Block Public Access en todos los buckets.
- Activar **encriptación obligatoria SSE-KMS**.
- Crear reglas de AWS Config para detectar:
 - buckets públicos,
 - buckets sin versionado,
 - buckets sin cifrado.
- Activar **Amazon Macie** para detectar datos sensibles.

2. Seguridad de red

- Implementar **AWS Network Firewall** entre subredes privadas y públicas.
- Activar **VPC Flow Logs** en modo *Full*.
- Integrar con **GuardDuty** para análisis de tráfico malicioso.
- Implementar **Traffic Mirroring** para inspección avanzada.

Fase 2: Gestión de Identidades y Gobernanza (Semana 2 a 4)

1. Reestructuración de IAM

- Eliminar permisos wildcard "Action": "*".
- Crear políticas basadas en permisos mínimos por función.
- Activar **IAM Access Analyzer**.

2. Fortalecimiento de acceso

- MFA obligatorio en:
 - Cuenta root
 - Administradores
 - DevOps
- Rotación automática de claves cada 90 días.
- Prohibir Access Keys en usuarios humanos → usar IAM Identity Center.

3. Gobernanza con AWS Organizations

- Crear estructura multi-cuenta:
 - Security
 - Shared Services
 - Prod
 - Dev
- Implementar **Service Control Policies (SCP)**:
 - Bloquear regiones no autorizadas.
 - Bloquear eliminación de CloudTrail.
 - Restringir IAM excesivo.

Fase 3: Seguridad de Aplicaciones (Semana 4 a 8)

1. Migración a DevSecOps

Incorporar seguridad en CI/CD con herramientas:

- **SAST** → SonarQube / GitHub Security
- **SCA** → Snyk / Dependency-Check
- **DAST** → OWASP ZAP en ambiente staging
- **Scanner de contenedores** → Amazon Inspector

2. Eliminación de credenciales hardcodeadas

- Integrar todas las apps con **AWS Secrets Manager** o Parameter Store.
- Habilitar rotación automática de secretos.

3. Validación segura de entradas

- Implementar sanitización obligatoria con librerías seguras.
- Aplicar OWASP Input Validation Cheat Sheet.

Fase 4: Detección y Respuesta continua (Semana 8 en adelante)

Monitoreo y detección

- Activar **AWS Security Hub** como centro de seguridad.
- Integrar:
 - GuardDuty
 - Macie
 - Inspector
 - Config

Alertas

- Configurar alarmas CloudWatch por:
 - cambios de IAM críticos,
 - creación de claves nuevas,
 - accesos fuera del horario,
 - accesos desde países no permitidos.

Respuesta a incidentes

Crear un SOP (procedimiento) que incluya:

- detección inicial,
- aislamiento de recursos,
- análisis del incidente,
- comunicación al equipo,
- recuperación,
- post-mortem.

Puedo prepararte ese documento también si querés.

3.4 Estrategia de Prioridad (para justificar ante profesor)

Prioridad	Acción	Justificación
Alta	Bloqueo de buckets públicos y IAM mínimo	Mitigan riesgo inmediato de filtración
Alta	MFA obligatorio	Evita compromisos críticos con robo de credenciales
Media	Network Firewall	Reduce movimiento lateral
Media	DevSecOps	Previene vulnerabilidades futuras
Baja	Red Team anual	Mejora continua pero no urgente

3.5 Resultado Esperado

Al implementar este plan:

- La superficie de ataque se reduce drásticamente.
- Los datos sensibles quedan protegidos con cifrado y acceso mínimo.
- IAM pasa de ser un riesgo a ser un sistema de control robusto.
- La red bloquea ataques internos y externos.
- Las aplicaciones dejan de llevar vulnerabilidades al entorno productivo.
- La empresa cumple buenas prácticas internacionales:
 - AWS Well-Architected
 - OWASP
 - CIS Benchmarks

3. Conclusiones

La seguridad en la nube debe abordarse como un proceso integral que abarque infraestructura, identidades, desarrollo seguro y monitoreo continuo. A través de la auditoría realizada se identificaron debilidades críticas que comprometían la confidencialidad y disponibilidad de los recursos, principalmente por configuraciones inseguras en S3, falta de segmentación en la red, permisos excesivos en IAM y ausencia de controles de seguridad en el ciclo de desarrollo.

Las soluciones propuestas permiten construir una arquitectura más resiliente mediante capas de defensa complementarias: políticas estrictas de acceso en S3, inspección profunda de tráfico con AWS Network Firewall, gobernanza de identidades con IAM Access Analyzer y MFA, y un pipeline DevSecOps que incorpora pruebas automáticas y manuales de seguridad. Además, se estableció un plan de implementación progresivo que facilita la adopción de controles sin afectar la operación del negocio.

Con estas mejoras, la organización adquiere una postura de seguridad madura, alineada con estándares como AWS Well-Architected, CIS Benchmarks y OWASP, reduciendo exposición a amenazas internas y externas, y garantizando la protección de datos críticos en un entorno de nube dinámico y en crecimiento.

4. Referencias

- Amazon Web Services. (2025). *AWS Security Best Practices*. <https://docs.aws.amazon.com>
- Amazon Web Services. (2025). *AWS Well-Architected Framework – Security Pillar*. <https://aws.amazon.com/architecture/well-architected/>
- Lucid Software Inc. (2025). *Lucidchart: Herramienta de diagramación y visualización de arquitectura*. <https://www.lucidchart.com>

